Complete finite validation programs for Topic F

The human_compression project

Canonical Topic F: validation appendix

The uniform written proofs do not invoke finite enumeration. The three complete programs below validate the coordinates, markings, exact source presentation and subgroup intersections at the parameters recorded in the replay report. Their complete text is retained, including comments and negative controls. The first two were authored with the original recovery; the third was independently authored for its accepted review. The two complete historical operational drivers are supplied separately. They are not additional mathematical proofs, and the historical review driver still requires its original archived package layout.

1 Mixed-modulus cyclotomic models

```
#!/usr/bin/env python3
"""Exact mixed-modulus cyclotomic models; emits literal GAP permutations."""
import argparse, itertools, json, math, pathlib

PARAMETERS = [(3,3),(3,4),(3,6),(5,3),(5,5),(7,3),(11,3)]

def need(ok, msg):
    if not ok:
        raise RuntimeError('PHASE236_REJECT '+msg)

def model(p, m, corrupt=False):
    d=p-1; n=m-1; e=(n-1)//d+1; r=(n-1)%d+1
    mod=[p**(e if i<r else e-1) for i in range(d)]
    def norm(v): return tuple(x%q for x,q in zip(v,mod))
    zero=norm([0]*d); one=norm([1]+[0]*(d-1))
    def add(v,w):return norm([x+y for x,y in zip(v,w)])
    def neg(v):return norm([-x for x in v])
    def act(T,v):return norm([sum(a*b for a,b in zip(row,v)) for row in T])
    def matrix(cols):return tuple(tuple(cols[j][i] for j in range(d)) for i in range(d))
    basis=[norm([int(i==j) for i in range(d)]) for j in range(d)]
    I=matrix(basis)
    def compose(T,U):return matrix([act(T,col) for col in zip(*U)])
    def power(T,k):
        out=I
        for _ in range(k):out=compose(T,out)
        return out
    pi=[[0]*d for _ in range(d)]
    for i in range(d-1):pi[i+1][i]=1
    for i in range(d):pi[i][-1]=-math.comb(p,i+1)
    if corrupt:pi[0][-1]=0
    Z=matrix([add(act(pi,v),v) for v in basis])
    def well_defined(T):
        return all((mod[j]*T[i][j])%mod[i]==0 for i in range(d) for j in range(d))
    need(well_defined(Z),'zeta respects the mixed-modulus lattice')
    need(power(Z,p)==I,'cyclotomic p-th power')
    need(Z!=I,'nonabelian action')
    Zinv=power(Z,p-1)
    barpi=matrix([add(act(Zinv,v),neg(v)) for v in basis])
    cols=[];v=one
    for _ in range(d):cols.append(v);v=act(barpi,v)
    S=matrix(cols)
    need(well_defined(S),'conjugation respects the mixed-modulus lattice')
    need(compose(S,S)==I,'conjugation square')
    need(compose(compose(S,Z),S)==Zinv,'conjugation reverses zeta')
```

```
    for v in basis:
        w=zero;u=v
        for _ in range(p):w=add(w,u);u=act(Z,u)
        need(w==zero,'cyclotomic norm zero')
    A=list(itertools.product(*(range(q) for q in mod)))
    points=[(v,k) for v in A for k in range(p)]
    idx={v:i+1 for i,v in enumerate(points)}
    need(len(points)==p**m,'normal-form cardinality')
    backwards=[act(power(Z,(-k)%p),one) for k in range(p)]
    s=[idx[(add(v,backwards[k]),k)] for v,k in points]
    beta=[idx[(v,(k+1)%p)] for v,k in points]
    sigma=[idx[(act(S,v),(-k)%p)] for v,k in points]
    tau=[idx[(neg(v),k)] for v,k in points]
    for perm in [s,beta,sigma,tau]:need(sorted(perm)==list(range(1,len(points)+1)),'literal
     ↪  permutation')
    return dict(p=p,m=m,e=e,r=r,moduli=mod,degree=len(points),s=s,beta=beta,sigma=sigma,tau
     ↪  =tau)

def emit(path, corrupt=False):
    data=[model(p,m,corrupt=corrupt) for p,m in PARAMETERS]
    rows=[]
    for row in data:
        fields=['p:='+str(row['p']),'m:='+str(row['m']),
                'moduli:='+str(row['moduli'])]
       fields += [key+':=PermList('+str(row[key])+')' for key in ['s','beta','sigma','tau']]
        rows.append('rec('+','.join(fields)+')')
    path.write_text('PHASE236_DATA := [\n'+',\n'.join(rows)+'\n];\n')
    return [{k:v for k,v in row.items() if k not in ['s','beta','sigma','tau']} for row in
     ↪  data]

if __name__=='__main__':
    ap=argparse.ArgumentParser(description=__doc__);ap.add_argument('--out',type=pathlib.Pa
     ↪  th,required=True)
    ap.add_argument('--corrupt-cyclotomic',action='store_true');a=ap.parse_args()
    need(not a.out.exists() and not a.out.is_symlink(),'fresh model output')
    print(json.dumps(emit(a.out,a.corrupt_cyclotomic),indent=2))
```

2 Marked-group and intersection validation

```
Need236:=function(ok,msg) if not ok then Error(Concatenation("PHASE236_REJECT ",msg)); fi;
 ↪  end;
IP236:=function(rr)
  local subsets,hs,I,J;
  subsets:=Combinations([1..3]); hs:=List(subsets,I->Group(List(I,i->rr[i]),()));
  for I in [1..8] do for J in [1..8] do
    if Intersection(hs[I],hs[J])<>hs[Position(subsets,Intersection(subsets[I],subsets[J]))]
     ↪  then
      return false;
    fi;
  od;od;return true;
end;
Run236:=function()
local row,p,m,s,beta,sigma,tau,P,A,ss,i,u,v,rr,G,par,a,b,points,x,y,r0,r1,r2;
for row in PHASE236_DATA do
  p:=row.p; m:=row.m; s:=row.s; beta:=row.beta; sigma:=row.sigma; tau:=row.tau;
  P:=Group(s,beta); A:=Subgroup(P,Orbit(Group(beta),s,OnPoints));
  Need236(Size(A)=p^(m-1) and IsAbelian(A),"abelian lattice order");
  Need236(Size(P)=p^m and not IsAbelian(P),"nonabelian kernel order");
  Need236(Order(beta)=p and Order(sigma)=2 and Order(tau)=2 and
   ↪  Comm(sigma,tau)=(),"complement");
  Need236(s^sigma=s and beta^sigma=beta^-1 and s^tau=s^-1 and beta^tau=beta,"source
   ↪  automorphisms");
  ss:=[s]; for i in [2..p-1] do Add(ss,Comm(ss[i-1],beta)); od;
  Need236(List(ss,Order)=row.moduli,"every source generator order");
  for u in ss do for v in ss do Need236(Comm(u,v)=(),"source abelian relators");od;od;
  Need236(Comm(ss[p-1],beta)=Product([1..p-1],i->ss[i]^(-Binomial(p,i))),"last source
   ↪  commutator");
  rr:=[s*tau*sigma,beta*tau*sigma,sigma]; G:=Group(rr);
  Need236(ForAll(rr,x->Order(x)=2),"marked involutions");
  Need236(Comm(rr[1],rr[3])=(),"string relation");
  Need236([Order(rr[1]*rr[2]),Order(rr[2]*rr[3])]=[p,2*p],"type");
```

```
  Need236(Size(G)=4*p^m,"full generation and order");
  Need236(IP236(rr),"all 64 intersection equalities");
  Need236(NilpotencyClassOfGroup(P)=m-1,"maximal class");
  Print("CYCLOTOMIC p=",p," m=",m," order=",Size(G)," moduli=",row.moduli," IP=64 PASS\n");
od;
# Independent affine models for the abelian classification, including unequal exponents.
for par in [[3,1,1],[3,1,2],[3,2,2],[5,1,2]] do
  p:=par[1]; a:=par[2]; b:=par[3]; points:=Cartesian([0..p^b-1],[0..p^a-1]);
  x:=PermList(List(points,v->Position(points,[(v[1]+1) mod p^b,v[2]])));
  y:=PermList(List(points,v->Position(points,[v[1],(v[2]+1) mod p^a])));
  r0:=PermList(List(points,v->Position(points,[v[1],(-v[2]) mod p^a])));
  r2:=PermList(List(points,v->Position(points,[(-v[1]) mod p^b,(-v[2]) mod p^a])));
  r1:=x*y*r2;
  rr:=[r0,r1,r2]; G:=Group(rr);
  Need236(ForAll(rr,r->Order(r)=2),"abelian marked involutions");
  Need236(Size(G)=4*p^(a+b),"abelian group order");
  Need236([Order(r0*r1),Order(r1*r2)]=[2*p^a,p^b],"abelian type");
  Need236(IP236(rr),"abelian intersection property");
  Need236((x*y*r0)^2=x^2 and x^2<>(),"literal source marking defect");
  Print("ABELIAN p=",p," a=",a," b=",b," order=",Size(G)," IP=64 BAD_MARKING_REJECTED\n");
od;
Print("PHASE236_FINITE_PASS\n");
end;
Run236();
QUIT;
```

3 Independent literal-presentation validation

```
# Independent literal source-presentation replay, without cyclotomic matrices.
Check248:=function(ok,s) if not ok then Error(Concatenation("P248_REJECT ",s));fi;end;;
IP248:=function(rr)
 local subsets,hs,i,j;
 subsets:=Combinations([1..3]);hs:=List(subsets,x->Group(rr{x},One(rr[1])));
 for i in [1..8] do for j in [1..8] do
  if Intersection(hs[i],hs[j])<>hs[Position(subsets,Intersection(subsets[i],subsets[j]))]
   ↪  then return false;fi;
 od;od;return true;
end;;
Run248:=function()
 local par,p,m,n,e,r,F,x,b,rels,i,j,U,iso,P,ss,beta,sigma,tau,K,G,a,z,rs,moduli;
 for par in [[3,3],[3,6],[5,3],[5,5],[11,3]] do
  p:=par[1];m:=par[2];n:=m-1;e:=QuoInt(n-1,p-1)+1;r:=((n-1) mod (p-1))+1;
  F:=FreeGroup(p);x:=GeneratorsOfGroup(F){[1..p-1]};b:=GeneratorsOfGroup(F)[p];
  moduli:=Concatenation(List([1..r],i->p^e),List([r+1..p-1],i->p^(e-1)));
  rels:=Concatenation(List([1..p-1],i->x[i]^moduli[i]),[b^p]);
  for i in [1..p-1] do for j in [i+1..p-1] do if j<=p-1 then
   ↪  Add(rels,Comm(x[i],x[j]));fi;od;od;
  for i in [1..p-2] do Add(rels,Comm(x[i],b)*x[i+1]^-1);od;
  Add(rels,Comm(x[p-1],b)*Product([1..p-1],i->x[i]^Binomial(p,i)));
  U:=F/rels;Check248(Size(U)=p^m,"exact presented order");
  iso:=IsomorphismPermGroup(U);Check248(iso<>fail,"permutation isomorphism");P:=Image(iso);
  ss:=List(GeneratorsOfGroup(U){[1..p-1]},x->Image(iso,x));beta:=Image(iso,GeneratorsOfGrou⌋
   ↪  p(U)[p]);
  Check248(List(ss,Order)=moduli,"individual source orders");
  sigma:=GroupHomomorphismByImages(P,P,[ss[1],beta],[ss[1],beta^-1]);
  tau:=GroupHomomorphismByImages(P,P,[ss[1],beta],[ss[1]^-1,beta]);
  Check248(sigma<>fail and tau<>fail and IsBijective(sigma) and IsBijective(tau),"literal
   ↪  maps extend to automorphisms");
  K:=Group(sigma,tau);Check248(Size(K)=4,"Klein complement");
  G:=SemidirectProduct(K,P);a:=Embedding(G,1);z:=Embedding(G,2);
  rs:=[Image(z,ss[1])*Image(a,tau)*Image(a,sigma),Image(z,beta)*Image(a,tau)*Image(a,sigma)⌋
   ↪  ,Image(a,sigma)];
  Check248(Size(Group(rs))=4*p^m and ForAll(rs,x->Order(x)=2),"marked generation");
  Check248([Order(rs[1]*rs[2]),Order(rs[2]*rs[3])]=[p,2*p],"exact type");
  Check248(Comm(rs[1],rs[3])=One(G) and IP248(rs),"all marked intersections");
  Print("LITERAL_PRESENTATION ",p," ",m," ORDER ",Size(U)," MODULI ",moduli," IP64 PASS\n");
 od;
 # The prime-index hypothesis is necessary: a composite-index subgroup can
 # meet another subgroup nontrivially without being contained in it.
 G:=ElementaryAbelianGroup(8);x:=GeneratorsOfGroup(G);P:=Group(x{[1,2]});K:=Group(x{[1,3]});
 Check248(Index(P,TrivialSubgroup(P))=4 and not IsSubgroup(K,P) and
  ↪  Size(Intersection(P,K))=2,"composite-index false weakening");
```

```
 Print("COMPOSITE_INDEX_FALSE_WEAKENING_REJECTED\nPHASE248_PRESENTATION_PASS\n");
end;;
Run248();
QUIT_GAP(0);
```